

SaaS in the news



BIZ & IT TECH SCIENCE POLICY CARS GAMING & CULTURE STORE FORUMS

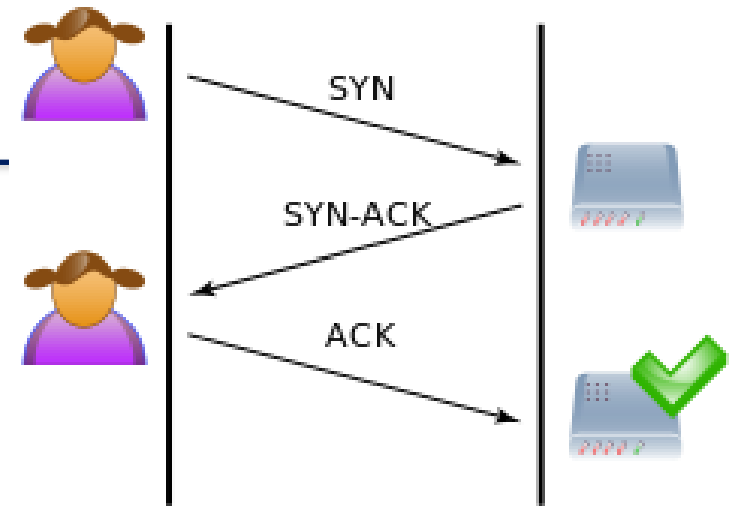
DDOS MULTIPLEXING —

Biggest DDoSes of all time generated by protocol 0-day in HTTP/2

More than 8 years after the adoption of HTTP/2, DDoSers devise rapid reset attack.

DAN GOODIN - 10/13/2023, 5:50 AM

As the Google post explained:

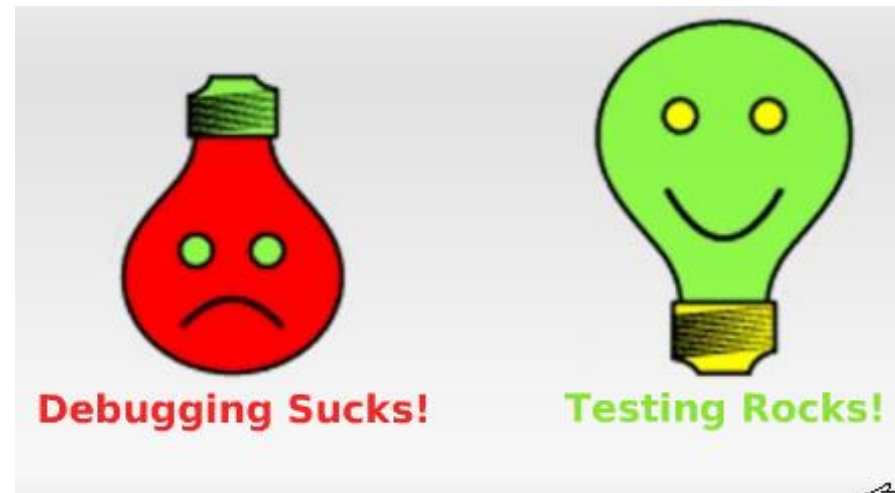


“

The ability to reset streams immediately allows each connection to have an indefinite number of requests in flight. By explicitly canceling the requests, the attacker never exceeds the limit on the number of concurrent open streams. The number of in-flight requests is no longer dependent on the round-trip time (RTT), but only on the available network bandwidth.

In a typical HTTP/2 server implementation, the server will still have to do significant amounts of work for canceled requests, such as allocating new stream data structures, parsing the query and doing header decompression, and mapping the URL to a resource. For reverse proxy implementations, the request may be proxied to the backend server before the RST_STREAM frame is processed. The client on the other hand paid almost no costs for sending the requests. This creates an exploitable cost asymmetry between the server and the client.

Another advantage the attacker gains is that the explicit cancellation of requests immediately after creation means that a reverse proxy server won't send a response to any of the requests. Canceling the requests before a response is written reduces downlink (server/proxy to attacker) bandwidth.



The Big Picture: FIRST, TDD, and Red-Green-Refactor

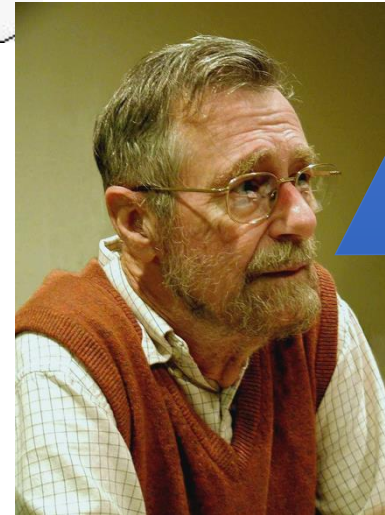
ESaaS §8.1

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License





Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it.



Testing can never demonstrate the absence of errors in software, only their presence

BDD+TDD: The Big Picture

- Behavior-driven design (BDD)
 - develop user stories (the features you wish you had) to describe how app will work
 - via Cucumber, user stories become acceptance tests and integration tests
- Test-driven development (TDD)
 - new code (both in app & possibly in step defs)
 - TDD: write tests before the new code itself
 - in other words: write tests for code you wish you had
- Deep insight: TDD improves modularity

Test-Driven development

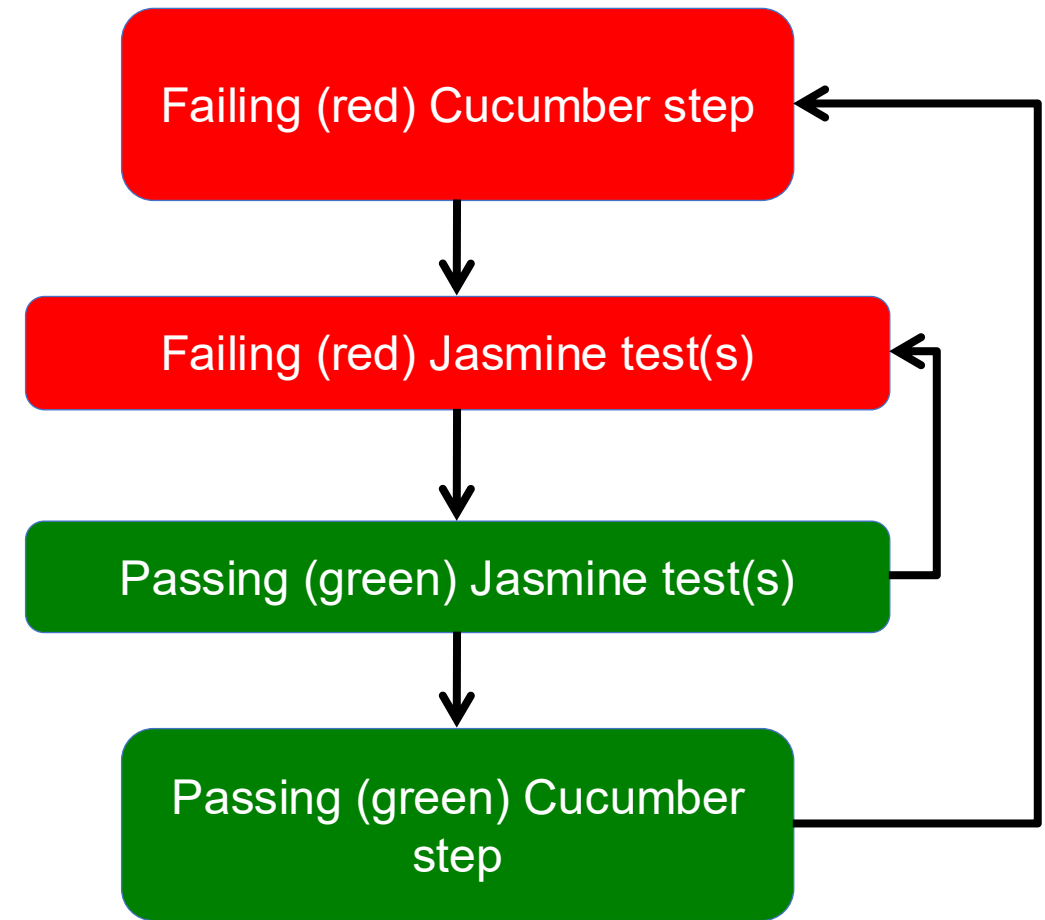
- Think about one thing the code should do
- Capture that thought in a test, which fails
- Write the simplest possible code that lets the test pass
- Refactor: DRY out commonality w/other tests
- Continue with next thing code should do

Red – Green – Refactor

Aim for “always have working code”

Cucumber.js & Jasmine

- Cucumber describes behavior via features & scenarios (behavior driven design)
- Jasmine tests individual modules that contribute to those behaviors (test driven development)



Testing Today: FIRST

ESaaS §8.1

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Testing Today

- Before: “debugging focus”
 - developers finish code, some ad-hoc testing
 - “toss over the wall to Quality Assurance [QA]”
 - QA staff manually poke at software
- Today: “maintainability & validation focus”
 - testing is part of every Agile iteration
 - developers test their own code
 - testing tools & processes highly automated
 - QA/testing group improves testability & tools
 - Still some manual testing too...but for different reasons

Unit tests should be FIRST

- **F**ast
- **I**ndependent
- **R**epeatable
- **S**elf-checking
- **T**imely

Unit tests should be FIRST

- **Fast**: run (subset of) tests quickly (since you'll be running them all the time)
- **Independent**: no tests depend on others, so can run any subset in any order
- **Repeatable**: run N times, get same result (to help isolate bugs and enable automation)
- **Self-checking**: test can automatically detect if passed (no human checking of output)
- **Timely**: written about the same time as code under test (with TDD, written first!)

Repeatable & Independent?

Which kinds of code can be tested
Repeatably and Independently?

- Code that relies on randomness (e.g. shuffling a deck of cards)?
- Code that relies on time of day (e.g. run backups every Sunday at midnight)?

Anatomy of a Test Case: Arrange, Act, Assert

ESaaS §8.2

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



You've seen this before

- **Arrange** precondition(s)
 - what about non-leaf methods?
 - what about methods that depend on time/randomness/env vars or an external HTTP service?
 - what about database/repository data used by the test?
 - what about “world state”(e.g. authenticated or not - session/JWT/middleware)?
- **Act** on the system under test (SUT)
 - what about testing route handlers/controllers? (Express/Koa/Fastify)
- **Assert** postcondition(s)
 - Pure function/service tests are “easy”...
 - ...but isolating handler/service/client even though a real HTTP request touches all layers?

Great expectations (assertions)

```
expect(term).toEqual("hardware")
```

Matcher

```
expect(results).toBeTruthy()
```

```
expect(title).toMatch(/hard/i)
```

```
expect(movie).toEqual
```

```
(jasmine.objectContaining
```

```
{ title: jasmine.any(String) })
```

Custom matchers (Jasmine) :
define your own matcher via
`jasmine.addMatchers(...)`

```
expect(results.length).toBe(0)
```

e.g. empty string? null/undefined? validation passes?

```
expect(() => newMovie.save()).toThrowError();
```

e.g. `ValidationError` / search term required

Great expectations (assertions)

```
const before = await Movie.count();  
await Movie.destroy(movieId);  
const after = await Movie.count();  
expect(after - before).toBe(-1);
```


Isolating Code: Doubles & Seams Intro

ESaaS §8.3

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Let's do a new feature

Feature: look up movie in TMDb

As a lazy moviegoer

So that I can add movies to the database
without filling in their info manually

I want to look up a movie by title in TMDb,
and populate RP from that info

Scenario: look up movie and 1 or more titles
match

Scenario: look up movie but no titles match

An aside: The code that contacts TMDb to search for a movie should be...?

1. A static method on `MovieService`?
2. An instance method on `Movie`?
3. Code in the route handler - `MoviesController.search`?

What does user story look like?

Given I am on the RottenPotatoes homepage
When I fill in "Search Terms" with "Inception"
And I follow "Search TMDb"
Then I expect to be on the RottenPotatoes homepage ...

So we need...

- A form with a search box on the main page
- A route handler to receive the form...
 - ... that reads `req.query.search_terms`
 - ... calls `MovieService.findInTMDb(search_terms)` ...
 - and renders the search results page ← new view
- And a route that maps `/movies/search` to the handler

The Code You Wish You Had

1. Call a service method that searches TMDb for the given title
2. If match found: render (new) "Search Results" page to display matches
3. If no match found: redirect to home page with message

Doubles & Seams: Jasmine & JavaScript

ESaaS §8.3

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Preview: Jasmine test helpers

- Additional helpers layered on Jasmine to test framework-specific things
 - e.g. call a route handler with fake req/res, or use supertest for GET/POST/...
 - a `res` “spy” to observe what the handler does (render/redirect/status/json)
- Matchers to test handler behavior, e.g.

```
expect(res.render).toHaveBeenCalledWith('movies/search',  
jasmine.any(Object));  
expect(res.render).toHaveBeenCalledWith('movies/search', jasmine  
.objectContaining({ results: jasmine.any(Array) }));
```
- Support for creating various *doubles* needed to test non-toy methods (later)

The code you wish you had

```
const moviesController = require('../controllers/moviesController');

describe('moviesController', () => {
  describe('searching TMDb', () => {
    it('calls the service method that performs TMDb search', async () => {
      const req = { query: { search_terms: 'hardware' } };
      const res = { render: jasmine.createSpy('render') };
      const movieService = { findInTMDB: jasmine.createSpy().and.resolveTo([]) };
      await moviesController.search(req, res, movieService);
      expect(movieService.findInTMDB).toHaveBeenCalled('hardware');
    });

    it('renders the Search Results page', async () => {});
    it('passes TMDb results to the view', async () => {});
  });
});
```


TDD for the Route Handler: Setup

- Add a route in the Express router
`router.get('/movies/search', moviesController.search);`
- Create an empty view/template for results:
`touch views/movies/search.ejs`
- Create a placeholder for the new handler:
`exports.search = async (req, res) => {
};`

What service method?

- Calling TMDb is responsibility of the service... but no model method exists to do this yet!

No problem...we'll fake the call to the code we wish we had:

`MovieService.findInTMDB(searchTerms)`

- Game plan:
 - Simulate submitting search form to route handler.
 - Check that the handler tries to call `MovieService.findInTMDB` with data from submitted form.
 - The test will fail (red), because the (empty) handler doesn't call `findInTMDB` yet.
 - Fix the handler to make green.

Seams

A **seam** is a place where you can modify the behavior of the app without changing its source code.



This is a method stub!

```
const moviesController = require('../controllers/moviesController');
```

```
describe('moviesController', () => {  
  describe('searching TMDb', () => {  
    it('calls the service method that performs TMDb search', async () => {  
      const movieService = { findInTMDb: jasmine.createSpy('findInTMDb') };  
      const req = { query: { search_terms: 'hardware' } };  
      const res = { render: jasmine.createSpy('render') };  
      await moviesController.search(req, res, movieService);  
      expect(movieService.findInTMDb).toHaveBeenCalledWith('hardware');  
    });  
    it('renders the Search Results page', async () => {});  
    it('passes TMDb results to the view', async () => {});  
  });  
});
```

Umm, wouldn't the real `findInTMDb` method actually return something??

```
exports.search = async (req, res, movieService) => {  
  const searchTerms = req.query.search_terms;  
  await movieService.findInTMDb(searchTerms);  
};
```

The Code You Wish You Had

1. Call a service method that searches TMDb for the given title
2. If match found: render (new) "Search Results" page to display matches
3. If no match found: redirect to home page with message

```
const moviesController = require('../controllers/moviesController');

describe('moviesController', () => {
  describe('searching TMDB', () => {
    it('calls the service method that performs TMDB search', async () => {
      const movieService = { findInTMDB: jasmine.createSpy('findInTMDB').and.resolveTo([]) };
      const req = { query: { search_terms: 'hardware' } };
      const res = { render: jasmine.createSpy('render') };
      await moviesController.search(req, res, movieService);
      expect(movieService.findInTMDB).toHaveBeenCalledWith('hardware');
    });
    it('renders the Search Results page', async () => {});
    it('passes TMDB results to the view', async () => {});
  });
});

exports.search = async (req, res, movieService) => {
  const searchTerms = req.query.search_terms;
  await movieService.findInTMDB(searchTerms);
};
```

“Shows results on
the Search Results
page”
is really 2 behaviors!

```

const moviesController = require('../controllers/moviesController');

describe('moviesController', () => {
  describe('searching TMDb', () => {
    it('calls the service method that performs TMDb search', async () => {
      const movieService = { findInTMDB: jasmine.createSpy('findInTMDB').and.resolveTo([]) };
      const req = { query: { search_terms: 'hardware' } };
      const res = { render: jasmine.createSpy('render') };
      await moviesController.search(req, res, movieService);
      expect(movieService.findInTMDB).toHaveBeenCalled('hardware');
    });
    it('renders the Search Results page', async () => {
      const movieService = { findInTMDB: jasmine.createSpy('findInTMDB').and.resolveTo([]) };
      const req = { query: { search_terms: 'hardware' } };
      const res = { render: jasmine.createSpy('render') };
      await moviesController.search(req, res, movieService);
      expect(res.render.calls.mostRecent().args[0]).toBe('search');
    });
    it('passes TMDb results to the view', async () => {});
  });
});

```

In Express handler tests, we assert on res.* calls, such as:

```

res.render(...)
res.redirect(...)
res.status(...)

```

Doubles & Seams: Method stubs & spies

ESaaS §8.3

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Where we are so far...

```
exports.search = async (req, res, movieService) => {  
  const searchTerms = req.query.search_terms;  
  await movieService.findInTMDB(searchTerms);  
};
```

The last spec says:

```
it('passes the TMDb search results to the view', async () => {});
```

How to make this spec green?

- Wouldn't the real `MovieService.findInTMDB` method actually return something?
 - *Well, what do we wish it would return?*

Remember!

- the handler gets a value from `findInTMDB`, and makes it available to the correct view
- We are not testing `findInTMDB`!

Making results available means passing locals to render

- In Express handler tests, we check what gets passed to the view via `res.render(...)`
 - the second argument to `res.render(view, locals)` contains the data for the template

```
expect(res.render).toHaveBeenCalledWith('search',  
    jasmine.objectContaining({ results: jasmine.any(Array) })))
```

```
exports.search = async (req, res, movieService) => {  
    const searchTerms = req.query.search_terms;  
    const results = await movieService.findInTMDB(searchTerms);  
    res.render('search', { results });  
};
```

Stunt doubles!

```
m = { title: 'Snowden' }
```

- To make it a bit smarter, **stub** some methods:

```
m = jasmine.createSpyObj('movie', ['title'])
```

```
m.title.and.returnValue('Snowden')
```

- Use when you need a stand-in with limited abilities
 - e.g.: you're passing it to another function that expects the object to respond to specific methods
 - Enable just enough behavior for the spec(s) that use it

Putting this all together...

```
it('passes search results to the view', async () => {
  const fakeResults = [
    { title: 'Fake Movie 1' },
    { title: 'Fake Movie 2' }
  ];
  const movieService = {
    findInTMDB: jasmine.createSpy('findInTMDB').and.resolveTo(fakeResults)
  };
  const req = { query: { search_terms: 'hardware' } };
  const res = { render: jasmine.createSpy('render') };
  await moviesController.search(req, res, movieService);
  expect(res.render).toHaveBeenCalledWith(
    'search',
    jasmine.objectContaining({ results: fakeResults })
  );
});
```

- The view receives results as an Array, and for this test, that's about all we care about!

Summary

System under test (SUT)	Testing strategy
Pure leaf function—no side effects, no collaborator methods or classes, same inputs yield same outputs	Assert correct output results for critical values and for arbitrary values in noncritical regions
Relies on results from calling collaborator methods or invoking behaviors on collaborator objects	In Arrange phase, create doubles that “force” the desired behavior by returning prearranged values, raising an exception, and so on
Nondeterministic or time-dependent behavior	In code under test, isolate the nondeterminism in a method call that can be stubbed using a double in the Arrange phase
Has side effects	In Arrange phase, observe the relevant state before executing test code; in Assert phase, observe it again and check for side effect

Figure 8.6: Strategy to properly isolate the SUT when it is not a pure function, not a leaf method, or both.

TDD Recap

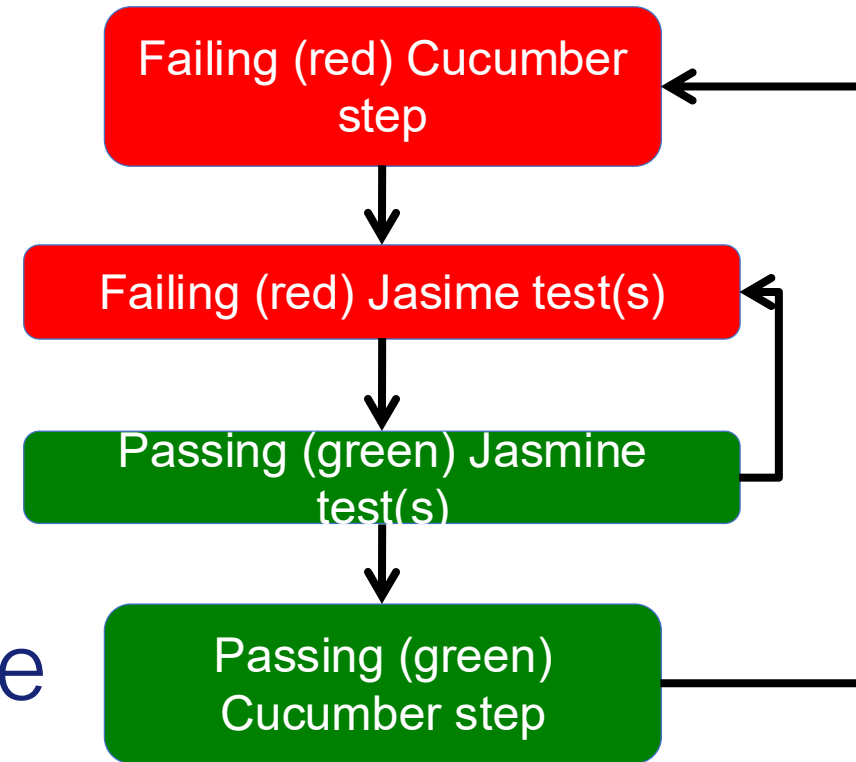
© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Where we started

Scenario: look up movie and 1 or more titles match

- Each spec tests 1 behavior
- **describe** groups related specs together
- **beforeEach()** captures preconditions common to a describe group



Writing the specs

- Each spec should test *just one behavior*
 - `describe` groups related specs together
- Use seams as needed to isolate that behavior
- Determine what type of expectation will check the behavior: what are you really testing?
- Write the test and make sure it fails for the right reason
- Add code to app until test passes green
- Look for opportunities to refactor/beautify

Unit vs. Functional tests in SaaS apps

- Unit tests: behavior within a method/class
 - collaborator classes are mocked
 - collaborator methods may be stubbed out (in this class or collaborator classes)
 - both are sometimes generically called doubles
- Functional test: behavior across methods/classes (multiple classes are exercised)
 - e.g. controller flow from GET/POST all the way to template rendering, which express.js stubs
 - (so mostly a true full-stack test, but for one request, not one workflow)

TDD vs. Debugging

ESaaS §8.12

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



TDD vs. Conventional debugging

Conventional	TDD
Write 10s of lines, run, hit bug: break out debugger	
Insert printf's to print variables while running repeatedly	
Stop in debugger, tweak/set variables to control code path	
Dammit, I thought for sure I fixed it, now I have to do this all again	

- Lesson 1: TDD uses same skills & techniques as conventional debugging—but more productive (FIRST)
- Lesson 2: writing tests before code takes more time up-front, but often less time overall

4. TDD & Testing

- “TDD is hard at first, but it gets easier”
- “Was great for quickly noticing bugs [regression] and made them easy to fix”
- “Helped me organize my thoughts as well as my code” [The code you wish you had]
- “Wish we had committed to it earlier & more aggressively”
- “We didn’t always test before pushing, and it caused a lot of pain”

5. Test coverage, code quality

- “Good coverage gave us confidence we weren’t breaking stuff when we deployed new code”
- “Felt great to get high grade from CodeClimate”
- “Pull-request model for constant code reviews made our code quality high”
- “Wish we had committed to TDD + coverage measurement earlier”

Fixtures & Factories

ESaaS §8.5

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



When you need the real thing

Where to get a real object:

- Fixture: statically preload some known data into database tables
- Factory: create only what you need per-test

```
describe("Movie", function () {
  it("returns name with rating", function () {
    const fake_movie = jasmine.createSpyObj("Movie", [
      "title",
      "rating",
      "name_with_rating",
    ]);

    // stub return values (like allow(...).to receive(...).and_return)
    fake_movie.title.and.returnValue("Casablanca");
    fake_movie.rating.and.returnValue("PG");

    // define behavior of name_with_rating
    fake_movie.name_with_rating.and.returnValue("Casablanca (PG)");

    expect(fake_movie.name_with_rating()).toBe("Casablanca (PG)");
  });
});
```


Fixtures

- Database wiped & reloaded with fixtures before each spec
- Pros/when to use
 - truly static data, e.g. configuration info that never changes
 - easy to see all test data in one place
- Cons/when not to use
 - may introduce dependency on fixture data

Factories

- Dynamic test data creation
 - Build objects with defaults and overrides
- Objects with relationships
 - Create parent/child structures (e.g., user → posts)
- Pros/when to use
 - objects are complex and reused across tests, variations of similar objects are needed, relationships between objects must be constructed,
- Cons/when not to use
 - simple objects where literals are clearer. data is small, static, and used only once,

Fixtures—example

</> JavaScript



```
describe("Movie", function () {
  it("returns name with rating", function () {
    const fake_movie = jasmine.createSpyObj("Movie", [
      "title",
      "rating",
      "name_with_rating",
    ]);

    // stub return values (like allow(...).to receive(...).and_return)
    fake_movie.title.and.returnValue("Casablanca");
    fake_movie.rating.and.returnValue("PG");

    // define behavior of name_with_rating
    fake_movie.name_with_rating.and.returnValue("Casablanca (PG)");

    expect(fake_movie.name_with_rating()).toBe("Casablanca (PG)");
  });
});
```

Example: Factory

</> JavaScript



```
// factory function
function movieFactory(overrides = {}) {
  return {
    title: "Casablanca",
    rating: "PG",
    ...overrides
  };
}

describe("Movie factory", function () {
  it("creates default movie", function () {
    const movie = movieFactory();

    expect(movie.title).toBe("Casablanca");
    expect(movie.rating).toBe("PG");
  });

  it("allows overrides", function () {
    const movie = movieFactory({ title: "Inception" });

    expect(movie.title).toBe("Inception");
    expect(movie.rating).toBe("PG"); // default still applies
  });
});
```

Factories with associations

</> JavaScript

```
// user factory
function userFactory(overrides = {}) {
  return {
    id: Math.floor(Math.random() * 1000),
    name: "Zhao",
    ...overrides
  };
}

// post factory with association
function postFactory(overrides = {}) {
  return {
    id: Math.floor(Math.random() * 1000),
    title: "Hello World",
    user: userFactory(), // association
    ...overrides
  };
}
```

</> JavaScript

```
describe("Post factory with association", function () {
  it("creates a post with a user", function () {
    const post = postFactory();

    expect(post.title).toBe("Hello World");
    expect(post.user).toBeDefined();
    expect(post.user.name).toBe("Zhao");
  });

  it("allows overriding the associated user", function () {
    const customUser = userFactory({ name: "Alice" });
    const post = postFactory({ user: customUser });

    expect(post.user.name).toBe("Alice");
  });
});
```

Fixtures

- Database wiped & reloaded with fixtures before each spec
- Pros/when to use
 - truly static data, e.g. configuration info that never changes
 - easy to see all test data in one place
- Cons/when not to use
 - may introduce dependency on fixture data

Factories

- Dynamic test data creation
 - Build objects with defaults and overrides
- Objects with relationships
 - Create parent/child structures (e.g., user → posts)
- Pros/when to use
 - objects are complex and reused across tests, variations of similar objects are needed, relationships between objects must be constructed,
- Cons/when not to use
 - simple objects where literals are clearer. data is small, static, and used only once,

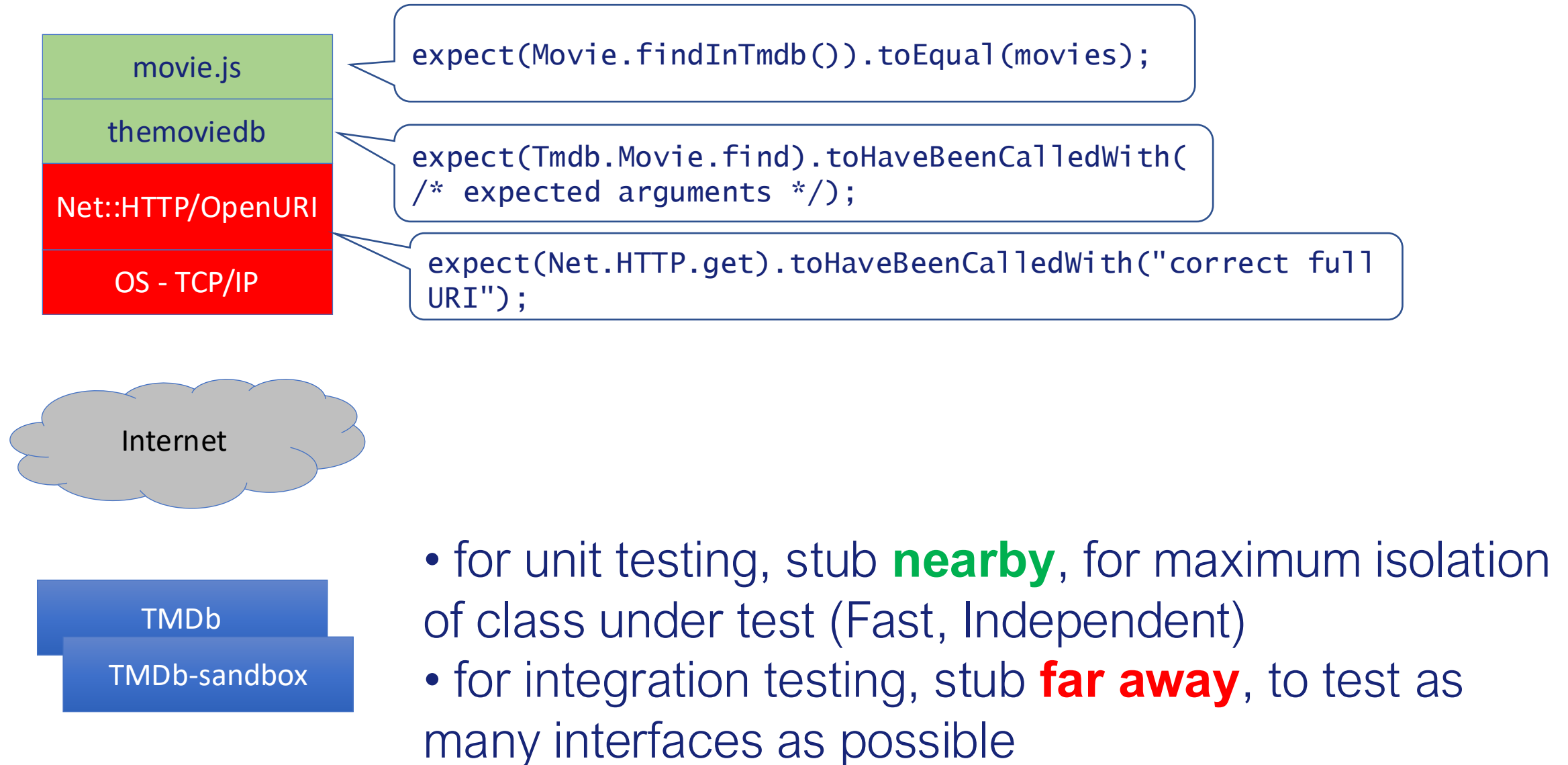
Stubbing the Internet

ESaaS §8.4

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Where to stub in SOA?



Coverage

ESaaS §8.7

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



How much testing is enough?

- Bad: “Until time to ship”
- A bit better: (Lines of test) / (Lines of code)
 - 1.2–1.5 not unreasonable
 - often much higher for production systems
- Better question: “How thorough is my testing?”
 - Formal methods
 - Coverage measurement
 - We focus on the latter, though the former is gaining steady traction

What does “100% coverage” mean?

- S0: every method
- S1: every method from every call site
- ➔ C0: every line touched
- C1: every branch in both directions
- C1+decision coverage: every subexpression in conditional
- C2: every path (difficult, and disagreement on how valuable)

```
class MyClass
  def foo(x,y,z)
    if x
      if (y && z) then bar(0) end
    else
      bar(1)
    end
  end
end
def bar(x)
end
```

if x is true in line 3 but y&&z is false in line 4, we increase C0 coverage but not C1 coverage

What kinds of tests?

- Unit (one method/class)
- Functional or module (a few methods/classes)
- Integration/system



Runs faster
Finer resolution
Many mocks;
Doesn't test interfaces
Higher coverage

Lower coverage
Fewer mocks;
tests interfaces
Coarser resolution
Runs slower

Going to extremes

- ✗ “I kicked the tires, it works”
- ✗ “Don’t ship until 100% covered & green”
- ✓ use coverage to identify untested or undertested parts of code
- ✗ “Focus on unit tests, they’re more thorough”
- ✗ “Focus on integration tests, they’re more realistic”
- ✓ each finds bugs the other misses

Other Testing Concepts

ESaaS §8.8

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Other testing terms you may hear

- Mutation testing: if introduce deliberate error in code, does some test break?
- Fuzz testing: 10,000 monkeys throw random input at your code
 - Find ~20% MS bugs, crash ~25% Unix utilities
 - Tests app the way it **wasn't** meant to be used
- DU-coverage: is every pair <define x/use x> executed?
- Black-box vs. white-box/glass-box

TDD Summary

- Red – Green – Refactor, and always have working code
- Test one behavior at a time, using seams
- Use it “placeholders” or pending to note tests you know you’ll need
- Read & understand coverage reports
- “Defense in depth”: don’t rely too heavily on any one kind of test

The Plan-and-Document Perspective on Testing

ESaaS §8.10

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



P&D Testing?

- BDD/TDD writes tests before code
 - When do P&D developers write tests?
- ...starts from user stories
 - Where do P&D developers start?
- ...developers write tests & code
 - Does P&D use same or different people for testing and coding?
- What does the Testing Plan and Testing Documentation look like?

P&D Project Manager

- P&D depends on Project Managers
- Document project management plan
- Creates Software Requirements Specification (SRS)
 - Can be 100s of pages
 - IEEE standard to follow
- Must document Test Plan
 - IEEE standard to follow



P&D Approach to Testing

- Manager divides SRS into programming units
- Developers code units, do unit testing
- Separate Quality Assurance (QA) team does higher level tests

3 QA Integration Options

- Top-down integration
 - Starts top of dependency graph
 - High-level functions (UI) work soon
 - Many stubs to get app to “work”
- Bottom-up integration
 - Start bottom of dependency graph
 - No stubs, integrate everything in a module
 - Can't see app working until all code written & integrated

3 Integration Options

- 3. Sandwich integration
 - Best of both worlds?
 - Reduce stubs by integrating some units bottom up
 - Try to get UI operational by integrating some units top down



Agile VS. P&D

<i>Tasks</i>	<i>In Plan-and-Document</i>	<i>In Agile</i>
Test Plan and Documentation	Software Test Documentation such as IEEE Standard 829-2008	User stories
Order of Coding and Testing	1. Code units 2. Unit test 3. Module test 4. Integration test 5. System test 6. Acceptance test	1. Acceptance test 2. Integration test 3. Module test 4. Unit test 5. Code units
Testers	Developers for unit tests; QA testers for module, integration, system, and acceptance tests	Developers
When Testing Stops	Company policy (e.g., statement coverage, happy and sad user inputs)	All tests pass (green)

Figure 8.15: The relationship between the testing tasks of Plan-and-Document versus Agile methodologies.